

# Aceleración del Método de Subgradiente: FISTA

- A mediados de los 2000 se desarrollaron ideas de este tipo en el área de reconstrucción de imágenes médicas, motivado por aplicaciones a Resonancia Nuclear Magnética.
- El algoritmo FISTA (Fast Iterative Shrinkage and Thresholding Algorithm) fue desarrollado por A. Beck y M. Teboulle, que implementa un concepto de aceleración similar al de Nesterov.
- Se aplica a un problema de la forma

$$\min_{x \in \mathbb{R}^n} \{g(x) + h(x)\}$$

donde  $g$  es no diferenciable pero  $h$  sí lo es, y  $\nabla h$  tiene propiedad de Lipschitz.

# El Método FISTA

- Describamos el método aplicado al problema

$$\min_{x \in \mathbb{R}^n} \tau \|x\|_1 + \|Ax - b\|_2^2$$

- Sea  $h(x) = \|Ax - b\|_2^2$
- El problema es, entonces:

$$\min_{x \in \mathbb{R}^n} \tau \|x\|_1 + h(x)$$

- Sea  $L$  constante de Lipschitz de  $\nabla h(x)$

# El Método FISTA

- **Fast Iterative Shrinkage-Thresholding Algorithm:**

**Data:**  $x^0, z^0 \in \mathbb{R}^n$ ,  $k = 0$ ,  $\theta_0 = 1$ .

**while** *Criterio de parada no se satisface* **do**

$y_k = z_k - \frac{1}{L} \nabla h(z_k)$ ;

    Resolver

$$\min_x \left\{ \tau \|x\|_1 + \frac{L}{2} \|x - y_k\|_2^2 \right\}$$

    Sea  $x_{k+1}$  la solución óptima. ;

    Sea  $\theta_{k+1} = \frac{1}{2} \left( 1 + \sqrt{1 + 4\theta_k^2} \right)$  .;

$z_{k+1} = x_{k+1} + \left( \frac{\theta_k - 1}{\theta_{k+1}} \right) (x_{k+1} - x_k)$  ;

$k = k + 1$ ;

**end**

- Recordemos:  $h(x) = \|Ax - b\|_2^2$

# El Método FISTA

- El problema

$$\min_x \left\{ \tau \|x\|_1 + \frac{L}{2} \|x - y_k\|_2^2 \right\}$$

no es realmente difícil de resolver...

# El Método FISTA

- El algoritmo tiene muy buen comportamiento para los problemas de LASSO y similares
- Su complejidad es la misma que para el método acelerado de Nesterov:
- Para llegar a una solución con error  $\epsilon$  se requieren

$$O\left(\frac{1}{\sqrt{\epsilon}}\right)$$

## En resumen:

- Tenemos las siguientes complejidades con métodos de primer orden (suponiendo  $\epsilon$  chico):
- Esto es el número de iteraciones para llegar a una solución  $\epsilon$ -óptima:

| Tipo de problema                           | Método Simple                           | Método Acelerado                          |
|--------------------------------------------|-----------------------------------------|-------------------------------------------|
| $f$ diferenciable, $\nabla f$ L-lipschitz  | $O\left(\frac{1}{\epsilon}\right)$      | $O\left(\frac{1}{\sqrt{\epsilon}}\right)$ |
| $f$ no diferenciable pero $f$ L-lipischitz | $O\left(\frac{1}{\epsilon^2}\right)$    | $O\left(\frac{1}{\sqrt{\epsilon}}\right)$ |
| $f$ fuerte convexa $\nabla f$ L-lipschitz  | $O\left(\log \frac{1}{\epsilon}\right)$ | $O\left(\log \frac{1}{\epsilon}\right)$   |

- El performance práctico puede ser significativamente mejor según el problema específico.
- **En teoría:** el método acelerado es “óptimo”, es lo mejor que se puede hacer con las propiedades que se tienen del problema.

# Problemas con restricciones

- Los problemas que hemos estudiado son no restringidos...
- Pero la idea de un Método de Primer Orden se puede extender a un problema con restricciones de forma muy directa.
- Consideremos un problema de la forma:

$$\begin{array}{ll} \text{mín} & f(x) \\ \text{s.a.} & x \in D \end{array}$$

donde  $D$  es convexo.

- Todo lo que hay que hacer es “proyectar” en  $D$ ...

# El método proyectado

- Tenemos el problema

$$\begin{array}{ll} \text{mín} & f(x) \\ \text{s.a.} & x \in D \end{array}$$

donde  $D$  es convexo.

- Sea  $\Pi_D(x)$  la proyección de  $x$  en  $D$ . Es decir:

$$\Pi_D(x) = \begin{array}{ll} \text{mín} & \|x - y\|_2 \\ \text{s.a.} & y \in D \end{array}$$

- El algoritmo se desarrolla adaptando la etapa de avance a:

$$x^{k+1} = \Pi_D(x^k + \lambda_k d^k)$$

donde  $d^k$  es la dirección de descenso y  $\lambda_k$  es un paso.

# El método proyectado

- En forma más específica, supongamos que  $L$  es la constante de Lipschitz de  $\nabla f$ .

- **Algoritmo simple de primer orden proyectado**

**Data:** Sea  $x^0 \in \mathbb{R}^n$ ,  $k = 0$ ,  $d^0 = -\nabla f(x^0)$ .

**while** *Criterio de parada no se satisface* **do**

$x^{k+1} = \Pi_D(x^k + \frac{1}{L}d^k);$   
     $d^{k+1} = -\nabla f(x^{k+1});$   
     $k = k + 1;$

**end**

## El método proyectado

- El algoritmo anterior es para una función  $f$  diferenciable, pero...
- También se puede proyectar el método de subgradiente, usando la etapa:

$$x^{k+1} = \Pi_D(x^k + \lambda_k d^k)$$

donde  $-d^k \in \partial f(x^k)$ .

- Las convergencias son iguales al caso sin restricciones y se usan los mismos  $\lambda$  para los pasos.
- Y también se puede acelerar!
- Pero la proyección requiere resolver un problema de optimización!!!!
- La idea anterior sólo sirve si el cálculo de  $\Pi_D(x)$ , la proyección, se pueda hacer en forma rápida.
- Esto ocurre cuando el conjunto  $D$  es “suficientemente simple”.

## El método proyectado: ejemplo

- Consideremos el siguiente problema (con  $a > 0$ ):

$$\begin{array}{ll} \text{mín} & \|Ax - b\|_2^2 \\ \text{s.a.} & \\ & -a \leq x_j \leq a, \quad j = 1, \dots, n \end{array}$$

- La función objetivo es convexa diferenciable y las restricciones son “simples”
- Aquí  $D = \{x : -a \leq x_j \leq a, \quad j = 1, \dots, n\}$ . ¿Cómo se proyecta sobre este conjunto?

## ¿Y si aún calcular el gradiente es muy complicado?

- Hemos mostrado métodos que no hacen “linesearch”, pero aún requieren el gradiente (o un subgradiente)
- Pero cuando  $n$  es realmente grande, incluso conocer eso puede ser imposible...
- O puede ser muy complejo evaluar el gradiente en sí.
- Por ejemplo, en el caso de la estimación de redes neuronales.
- Lo que se puede hacer en ese caso es, entonces, usar una aproximación al gradiente.
- Los métodos de este tipo caen dentro de los que se conoce como Métodos de Gradiente Estocástico (stochastic gradient).

## Pero estaríamos usando la dirección “equivocada” ...

- El impacto de no usar la dirección de gradiente puede no ser muy grande y se puede acotar.
- Específicamente, en la iteración  $k$ ,  $d^k$  es la dirección de avance real, pero usamos  $\tilde{d}^k$ .
- Entonces, los puntos de las iteraciones serían:

$$x^{k+1} = x^k - \lambda_k d^k, \quad \tilde{x}^{k+1} = x^k - \lambda_k \tilde{d}^k$$

- La desviación de la trayectoria de iteraciones es  $\|x^{k+1} - \tilde{x}^{k+1}\|$ .
- Entonces:

$$\|x^{k+1} - \tilde{x}^{k+1}\| = \lambda_k \|d^k - \tilde{d}^k\|$$

Pero estaríamos usando la dirección “equivocada” ...

- Tenemos:

$$\|x^{k+1} - \tilde{x}^{k+1}\| = \lambda_k \|d^k - \tilde{d}^k\|$$

- Luego, si, por ejemplo, el error en la dirección estuviera acotada de la forma:

$$\|d^k - \tilde{d}^k\| \leq \delta \|\nabla f(x^k)\|$$

para  $\delta$  “chico”, podemos argumentar que no nos desviaremos demasiado de la “trayectoria correcta” y deberíamos poder argumentar convergencia y complejidades similares al métodos exacto.

# Métodos de Gradiente Estocástico

- Por ejemplo, queremos resolver

$$P) \quad \min_{x \in \mathbb{R}^n} f(x)$$

- donde  $f$  tiene la siguiente forma:

$$f(x) = \frac{1}{p} \sum_{j=1}^p g_j(x)$$

donde  $p$  es muy, pero muy.... muy grande...

- Pero sí podemos calcular fácilmente  $\nabla g_j(x)$  para cualquier  $j$  particular.
- $f$  es una especie de “promedio”, entonces aproximamos  $\nabla f(x)$  por una “muestra” de sus sumandos:

$$\nabla f(x) \approx h_r(x) = \frac{1}{r} \sum_{j \in R} g_j(x)$$

donde  $R \subset \{1, \dots, p\}$  es una muestra de los índices totales, de tamaño  $r$ , “chico”.

# Métodos de Gradiente Estocástico

- Se tiene que

$$\mathbb{E}(h_r(x)) = \nabla f(x)$$

si la muestra es “representativa”.

- También se puede mostrar que si  $Var(h_r)$  no es “muy grande”, entonces se conservan las propiedades de convergencia.
- Hay teoría desarrollada, y práctica, que permiten determinar  $r$  adecuado y la forma de elegir la muestra.
- Esta idea se puede aplicar también a los subgradiientes, en el caso de un método no diferenciable.

# Métodos de Gradiente Estocástico

- Volvamos al problema

$$P) \quad \min_{x \in \mathbb{R}^n} f(x)$$

- y supongamos que  $n$  en sí es muy grande.
- En ese caso, hay que invertir mucho tiempo en calcular cada componente de  $\nabla f(x)$ , que son las derivadas parciales.
- Entonces: se propone muestrear  $R \subset \{1, \dots, n\}$  con tamaño “chico” y después usar como aproximación al gradiente el vector  $g_R(x)$  dado por:

$$g_R(x)_j = \begin{cases} \frac{\partial f}{\partial x_j}(x) & j \in R \\ 0 & j \notin R \end{cases}$$

(se usan sólo algunas de las derivadas parciales)

- Entonces, también se espera que  $g_R(x)$  sea una buena aproximación a  $\nabla f(x)$  y tenga buenas propiedades de descenso.
- Incluso, se puede considerar el tamaño de las derivadas en la muestra...

## Palabras al cierre del tema

- Presentamos una muy breve muestra de las ideas que se han desarrollado en Métodos de Primer Orden.
- Muchas descansan en algoritmos “viejos”, pero interpretados “a la moderna”
- Recuerden lo fundamental: no es el número de iteraciones en sí lo importante en la práctica sino el tiempo total, que depende del tiempo por iteración.
- Aquí es donde estos métodos tienen una ventaja, especialmente con problemas realmente grandes, y de ahí su importancia en temas de Ciencias de Datos e Inteligencia Artificial.